

1. Davida — Clients

Fichiers

```
model/Client.java
dto/ClientDTO.java
repository/ClientRepository.java
service/ClientService.java
controller/ClientController.java

views/commerce/clients.jsp
views/commerce/formClient.jsp
```

Model exact : Client.java

```
private Long id;
private String nom;
private String telephone;
private String adresse;
private LocalDateTime createdAt;
```

DTO exact : ClientDTO.java

```
private Long id;
private String nom;
private String telephone;
private String adresse;
```

Fonctions repository

```
List<Client> findAllOrderByNomAsc();

boolean existsByNomIgnoreCase(String nom);
```

Fonctions service

```
List<Client> findAll();

Client findById(Long id);

ClientDTO getForm(Long id);

String creer(ClientDTO dto);

String modifier(Long id, ClientDTO dto);

String valider(ClientDTO dto);
```

Routes controller

```
@GetMapping("/clients")
public String listClients(Model model);

@GetMapping("/clients/form")
```

```
public String showClientForm(@RequestParam(required = false) Long id, Model model);
```

```
@PostMapping("/clients/save")  
public String saveClient(@ModelAttribute ClientDTO dto, Model model);
```

Validations

nom obligatoire
telephone facultatif
adresse facultative

2. Fanilo — Ventes

Fichiers

model/Vente.java
model/DetailVente.java

dto/VenteDTO.java
dto/DetailVenteDTO.java

repository/VenteRepository.java
repository/DetailVenteRepository.java

service/VenteService.java

controller/VenteController.java

views/commerce/ventes.jsp
views/commerce/formVente.jsp
views/commerce/detailVente.jsp

Model exact : Vente.java

```
private Long id;  
private Client client;  
private LocalDate dateVente;  
private BigDecimal montantTotal;  
private String statut;  
private LocalDateTime createdAt;  
private LocalDateTime updatedAt;
```

Statuts exacts :

BROUILLON
VALIDEE
ANNULEE

Model exact : DetailVente.java

```
private Long id;  
private Vente vente;  
private LotPorc lot;  
private Integer quantite;  
private BigDecimal poidsTotal;  
private BigDecimal prixUnitaire;
```

```
private BigDecimal montant;
```

DTO : VenteDTO.java

```
private Long id;  
private Long clientId;  
private LocalDate dateVente;  
  
private Long lotId;  
private Integer quantite;  
private BigDecimal poidsTotal;  
private BigDecimal prixUnitaire;
```

DTO : DetailVenteDTO.java

```
private Long lotId;  
private Integer quantite;  
private BigDecimal poidsTotal;  
private BigDecimal prixUnitaire;  
private BigDecimal montant;
```

Fonctions repository

```
List<Vente> findAllByOrderByDateVenteDesc();  
  
List<Vente> findByDateVenteBetween(LocalDate debut, LocalDate fin);  
  
List<Vente> findByStatutOrderByDateVenteDesc(String statut);  
  
List<DetailVente> findByVenteId(Long venteId);
```

Fonctions service

```
List<Vente> findAll();  
  
Vente findById(Long id);  
  
VenteDTO getForm(Long id);  
  
String creerVente(VenteDTO dto);  
  
Créer une vente en statut :  
BROUILLON  
  
String validerVente(Long venteId);
```

Rôle :

1. vérifier que la vente existe
2. vérifier que le lot existe
3. vérifier quantité <= effectifActuel
4. passer la vente en VALIDEE
5. diminuer effectifActuel du lot
6. créer mouvement lot type VENTE

```
String annulerVente(Long venteId);
```

Rôle :

passer la vente en ANNULEE

```
BigDecimal calculerMontantTotal(VenteDTO dto);
```

Calcul :

montant = quantite × prixUnitaire

```
String validerDonneesVente(VenteDTO dto);
```

Validations :

```
clientId obligatoire  
lotId obligatoire  
quantite > 0  
prixUnitaire >= 0  
poidsTotal facultatif mais > 0 si renseigné
```

```
String creerMouvementVente(Long lotId, Integer quantite, Long venteId);
```

Type mouvement exact :

VENTE

Routes controller

```
@GetMapping("/ventes")  
public String listVentes(Model model);
```

```
@GetMapping("/ventes/form")  
public String showForm(@RequestParam(required = false) Long id, Model model);
```

```
@PostMapping("/ventes/save")  
public String save(@ModelAttribute VenteDTO dto, Model model);
```

```
@PostMapping("/ventes/valider/{id}")  
public String valider(@PathVariable Long id);
```

```
@PostMapping("/ventes/annuler/{id}")  
public String annuler(@PathVariable Long id);
```

```
@GetMapping("/ventes/{id}")  
public String detail(@PathVariable Long id, Model model);
```

3. Noah — Ingrédients

Fichiers

```
model/Ingredient.java  
dto/IngredientDTO.java  
repository/IngredientRepository.java  
service/IngredientService.java  
controller/IngredientController.java
```

```
views/stocks/ingredients.jsp
views/stocks/formIngredient.jsp
```

Model exact : Ingredient.java

```
private Long id;
private String nom;
private String unite;
private BigDecimal stockActuel;
private BigDecimal seuilAlerte;
private LocalDateTime createdAt;
private LocalDateTime updatedAt;
```

DTO exact : IngredientDTO.java

```
private Long id;
private String nom;
private String unite;
private BigDecimal stockActuel;
private BigDecimal seuilAlerte;
```

Fonctions repository

```
List<Ingredient> findAllOrderByNomAsc();

boolean existsByNomIgnoreCase(String nom);
```

Pour stock faible :

```
@Query("SELECT i FROM Ingredient i WHERE i.stockActuel <= i.seuilAlerte")
List<Ingredient> findStocksFaibles();
```

Fonctions service

```
List<Ingredient> findAllIngredients();

Ingredient findById(Long id);

IngredientDTO getForm(Long id);

String creerIngredient(IngredientDTO dto);

String modifierIngredient(Long id, IngredientDTO dto);

String validerIngredient(IngredientDTO dto);

boolean estStockFaible(Ingredient ingredient);

List<Ingredient> listerStocksFaibles();
```

Routes controller

```
@GetMapping("/ingredients")
public String listIngredients(Model model);

@GetMapping("/ingredients/form")
public String showForm(@RequestParam(required = false) Long id, Model model);
```

```
@PostMapping("/ingredients/save")
public String save(@ModelAttribute IngredientDTO dto, Model model);
```

Validations

```
nom obligatoire
unite facultative mais conseillée
stockActuel >= 0
seuilAlerte >= 0
```

4. Manoa — Mouvements de stock alimentaire

Fichiers

```
model/MouvementStockAliment.java
dto/MouvementStockDTO.java
repository/MouvementStockAlimentRepository.java
service/MouvementStockService.java
controller/MouvementStockController.java

views/stocks/mouvements.jsp
views/stocks/formMouvement.jsp
```

Model exact : MouvementStockAliment.java

```
private Long id;
private Ingredient ingredient;
private String typeMouvement;
private BigDecimal quantite;
private LocalDate dateMouvement;
private String observation;
private LocalDateTime createdAt;
```

Types exacts :

```
ENTREE
SORTIE
AJUSTEMENT
```

DTO exact : MouvementStockDTO.java

```
private Long id;
private Long ingredientId;
private String typeMouvement;
private BigDecimal quantite;
private LocalDate dateMouvement;
private String observation;
```

Fonctions repository

```
List<MouvementStockAliment> findAllOrderByDateMouvementDesc();

List<MouvementStockAliment> findByIngredientIdOrderByDateMouvementDesc(Long ingredientId);
```

Fonctions service

```
List<MouvementStockAliment> findAllMouvements();

MouvementStockDTO getForm();

String enregistrerMouvementStock(MouvementStockDTO dto);

ou directement :

String enregistrerMouvementStock(
    Long ingredientId,
    String typeMouvement,
    BigDecimal quantite,
    LocalDate dateMouvement,
    String observation
);

String validerMouvementStock(MouvementStockDTO dto);

boolean verifierStockDisponible(Long ingredientId, BigDecimal quantite);

String appliquerEntree(Ingredient ingredient, BigDecimal quantite);

String appliquerSortie(Ingredient ingredient, BigDecimal quantite);
```

Règles

```
ENTREE => stockActuel = stockActuel + quantite
SORTIE => stockActuel = stockActuel - quantite
SORTIE impossible si quantite > stockActuel
quantite > 0
```

Routes controller

```
@GetMapping("/stocks/mouvements")
public String listMouvements(Model model);

@GetMapping("/stocks/mouvements/form")
public String showForm(Model model);

@PostMapping("/stocks/mouvements/save")
public String saveMouvement(@ModelAttribute MouvementStockDTO dto, Model model);
```

5. Mandresy — Catégories dépenses + modèles finance

Fichiers

```
model/CategorieDepense.java
model/Depense.java

dto/CategorieDepenseDTO.java
dto/DepenseDTO.java
```

```
repository/CategorieDepenseRepository.java  
service/CategorieDepenseService.java
```

Model exact : CategorieDepense.java

```
private Long id;  
private String nom;
```

Model exact : Depense.java

```
private Long id;  
private CategorieDepense categorie;  
private LocalDate dateDepense;  
private BigDecimal montant;  
private String description;  
private LocalDateTime createdAt;
```

DTO exact : CategorieDepenseDTO.java

```
private Long id;  
private String nom;
```

DTO exact : DepenseDTO.java

```
private Long id;  
private Long categorieId;  
private LocalDate dateDepense;  
private BigDecimal montant;  
private String description;
```

Fonctions repository

```
List<CategorieDepense> findAllOrderByNomAsc();  
  
boolean existsByNomIgnoreCase(String nom);
```

Fonctions service

```
List<CategorieDepense> findAllCategories();  
  
CategorieDepense findById(Long id);  
  
String creerCategorie(CategorieDepenseDTO dto);  
  
String modifierCategorie(Long id, CategorieDepenseDTO dto);  
  
String validerCategorie(CategorieDepenseDTO dto);
```

Données initiales dans data.sql

Comme ta table contient seulement id et nom, il faut faire :

```
INSERT INTO categories_depenses(id, nom) VALUES  
(1, 'Alimentation'),  
(2, 'Santé'),  
(3, 'Personnel'),
```



```
(4, 'Transport'),  
(5, 'Maintenance'),  
(6, 'Autre');
```

6. Miaro — Dépenses

Fichiers

```
repository/DepenseRepository.java  
service/DepenseService.java  
controller/DepenseController.java
```

```
views/finance/depenses.jsp  
views/finance/formDepense.jsp
```

Fonctions repository

```
List<Depense> findAllOrderByDateDepenseDesc();  
  
List<Depense> findByDateDepenseBetween(LocalDate debut, LocalDate fin);  
  
List<Depense> findByCategorieId(Long categorieId);
```

Total dépenses :

```
@Query("SELECT COALESCE(SUM(d.montant), 0) FROM Depense d")  
BigDecimal totalDepenses();
```

```
@Query("""  
    SELECT COALESCE(SUM(d.montant), 0)  
    FROM Depense d  
    WHERE d.dateDepense BETWEEN :debut AND :fin  
""")  
BigDecimal totalDepensesEntre(LocalDate debut, LocalDate fin);
```

Fonctions service

```
List<Depense> findAllDepenses();  
  
Depense findById(Long id);  
  
DepenseDTO getForm(Long id);  
  
String enregistrerDepense(DepenseDTO dto);  
  
String modifierDepense(Long id, DepenseDTO dto);  
  
String validerDepense(DepenseDTO dto);  
  
BigDecimal calculerTotalDepenses();  
  
BigDecimal calculerTotalDepenses(LocalDate debut, LocalDate fin);
```

Validations

```
dateDepense obligatoire  
montant obligatoire  
montant > 0  
categorieId facultatif  
description facultative
```

Routes controller

```
@GetMapping("/depenses")  
public String listDepenses(Model model);  
  
@GetMapping("/depenses/form")  
public String showForm(@RequestParam(required = false) Long id, Model model);  
  
@PostMapping("/depenses/save")  
public String save(@ModelAttribute DepenseDTO dto, Model model);
```

7. Mickaella — Rapports financiers

Fichiers

```
dto/RapportFiltreDTO.java  
dto/RapportFinancierDTO.java  
  
service/RapportService.java  
controller/RapportController.java  
  
views/rapports/index.jsp
```

DTO : RapportFiltreDTO.java

```
private LocalDate dateDebut;  
private LocalDate dateFin;
```

DTO : RapportFinancierDTO.java

```
private BigDecimal totalVentes;  
private BigDecimal totalDepenses;  
private BigDecimal beneficeNet;  
  
private Long nombreVentes;  
private Long nombreDepenses;  
  
private LocalDate dateDebut;  
private LocalDate dateFin;
```

Fonctions service

```
RapportFinancierDTO genererRapport(RapportFiltreDTO filtre);  
  
BigDecimal calculerTotalVentes(LocalDate debut, LocalDate fin);
```

Important : compter seulement les ventes :

```
statut = VALIDEE
```

```
BigDecimal calculerTotalDepenses(LocalDate debut, LocalDate fin);
```

```
BigDecimal calculerBeneficeNet(BigDecimal totalVentes, BigDecimal  
totalDepenses);
```

```
Long compterVentes(LocalDate debut, LocalDate fin);
```

```
Long compterDepenses(LocalDate debut, LocalDate fin);
```

```
String validerFiltre(RapportFiltreDTO filtre);
```

Validations

```
dateDebut facultative
```

```
dateFin facultative
```

```
si les deux dates existent : dateDebut <= dateFin
```

Route controller

```
@GetMapping("/rapports")
```

```
public String index(@ModelAttribute RapportFiltreDTO filtre, Model model);
```

Model attendu :

```
model.addAttribute("filtre", filtre);
```

```
model.addAttribute("rapport", rapportService.genererRapport(filtre));
```

View

```
views/rapports/index.jsp
```

Afficher :

```
total ventes
```

```
total dépenses
```

```
bénéfice net
```

```
nombre ventes
```

```
nombre dépenses
```

```
filtre date début / date fin
```

8. Tsanta — Import / Export

Fichiers

```
model/ImportExport.java
```

```
dto/ImportExcelDTO.java
```

```
repository/ImportExportRepository.java
```

```
service/ImportExportService.java
```

```
controller/ImportExportController.java
```

```
views/imports/form.jsp
views/imports/historique.jsp
```

Model exact : ImportExport.java

```
private Long id;
private Utilisateur utilisateur;
private String typeOperation;
private String formatFichier;
private String module;
private String nomFichier;
private LocalDateTime dateOperation;
private String statut;
private String message;
```

Valeurs exactes :

```
typeOperation : IMPORT, EXPORT
formatFichier : EXCEL, PDF
statut : SUCCES, ECHEC
```

DTO : ImportExcelDTO.java

```
private MultipartFile file;
private String module;
private Long utilisateurId;
```

Fonctions repository

```
List<ImportExport> findAllByOrderByDateOperationDesc();
List<ImportExport> findByModuleOrderByDateOperationDesc(String module);
List<ImportExport> findByStatutOrderByDateOperationDesc(String statut);
```

Fonctions service

```
List<ImportExport> historique();

void tracerImport(
    String module,
    String nomFichier,
    String statut,
    String message,
    Long utilisateurId
);

void tracerExport(
    String module,
    String formatFichier,
    String nomFichier,
    String statut,
    String message,
    Long utilisateurId
);

String importerIngredientsExcel(MultipartFile file, Long utilisateurId);

String importerClientsExcel(MultipartFile file, Long utilisateurId);
```

```
byte[] exporterLotsExcel(Long utilisateurId);

byte[] exporterVentresExcel(Long utilisateurId);

byte[] exporterDepensesExcel(Long utilisateurId);

byte[] exporterRapportFinancierPdf(RapportFiltreDTO filtre, Long utilisateurId);
```

Routes controller

```
@GetMapping("/imports")
public String indexImports(Model model);

@GetMapping("/imports/form")
public String showImportForm(Model model);

@PostMapping("/imports/excel")
public String importerExcel(
    @RequestParam MultipartFile file,
    @RequestParam String module,
    Model model
);

@GetMapping("/exports/excel")
public void exporterExcel(
    @RequestParam String module,
    HttpServletResponse response
);

@GetMapping("/exports/pdf")
public void exporterPdf(
    @RequestParam String module,
    HttpServletResponse response
);

@GetMapping("/imports-exports/historique")
public String historique(Model model);

Tsanta      : import/export + historique
```